pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Randomized Search and Honest Tuning

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Week 3: the Polish rows, and today's question

Course 4's Polish training rows

4,680

training rows

326

bankrupt (6.97%)

Every fit, score, and choice by cross-validation inside these rows. Course 4's 1,170 test rows stay closed.

Today's question

A search always finds a best candidate. Is it better, or only the luckiest?

Primary metric: average precision, because the outcome is rare.

Allowed names

model probability of bankruptcy in this file

flagged at threshold t

Never called

a credit view on any company

a rating

How five-fold cross-validation works

1. Cut five folds

the bankrupt share the same in each

2. Fit on four

score the fifth

3. Repeat five times

every row scored once, unseen

4. Average

a mean to compare, an sd to compare it against

You control

The folds and the shuffle seed (mlkit.cv_folds), and the score.

What comes out

Five scores per model.

The default model, and what a grid costs

0.7783

default LightGBM, mean of 5 folds

0.0469

fold standard deviation

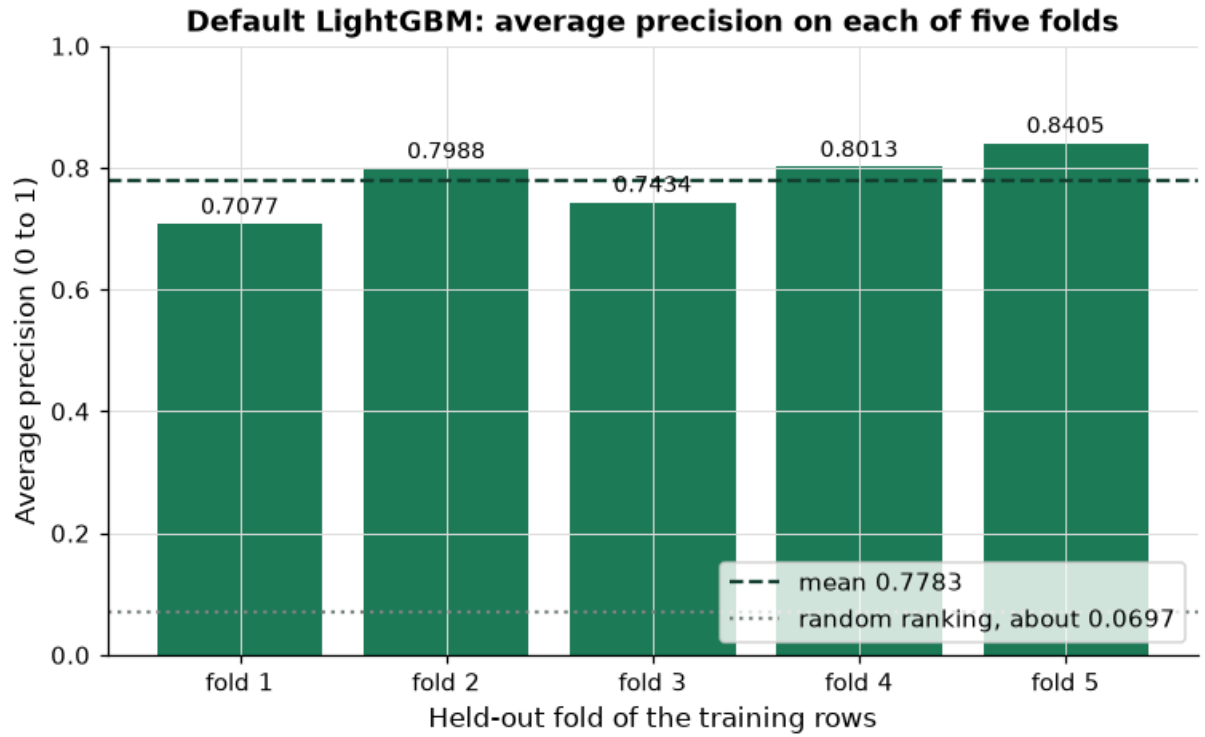
100 trees, 31 leaves, learning rate 0.1, at least 20 rows per leaf

Grid	Candidates	Models (5 folds)
2 values of 3 settings	8	40
5 values of 3 settings	125	625

Cost is candidates times folds, and each setting multiplies it.

A two-value grid only ever looks at two values of each setting.

One model, five fold scores



Same model, five scores

0.7077
fold 1

0.8405
fold 5

All five: 0.7077, 0.7988, 0.7434, 0.8013, 0.8405.
The spread is the rows, not the model.

Read every tuning gain today against bars like these.

How a hyperparameter search works

1. Write candidates

a list, a distribution, or a sampler

2. Fit each

one model per candidate

3. Score each

by the same five folds

4. Keep the best

its score is flattered by the choosing

You control

The candidates, the folds, and the score.

What comes out

One score per candidate, and the best row.

The ladder, its rows, and its search space

3,744

tuning rows, 261 bankrupt

936

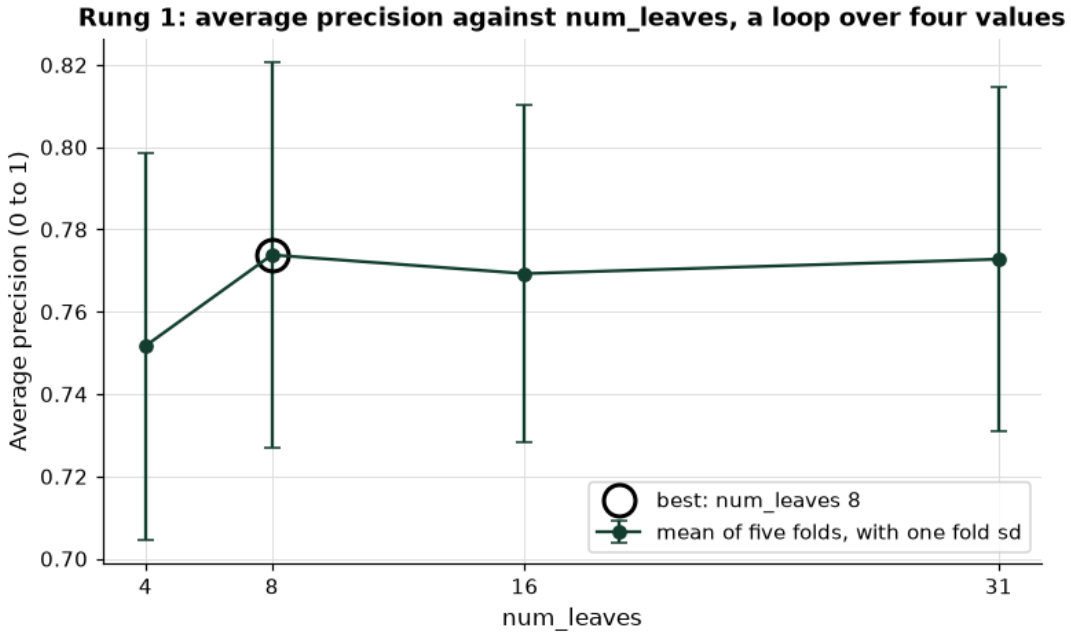
check rows, 65 bankrupt

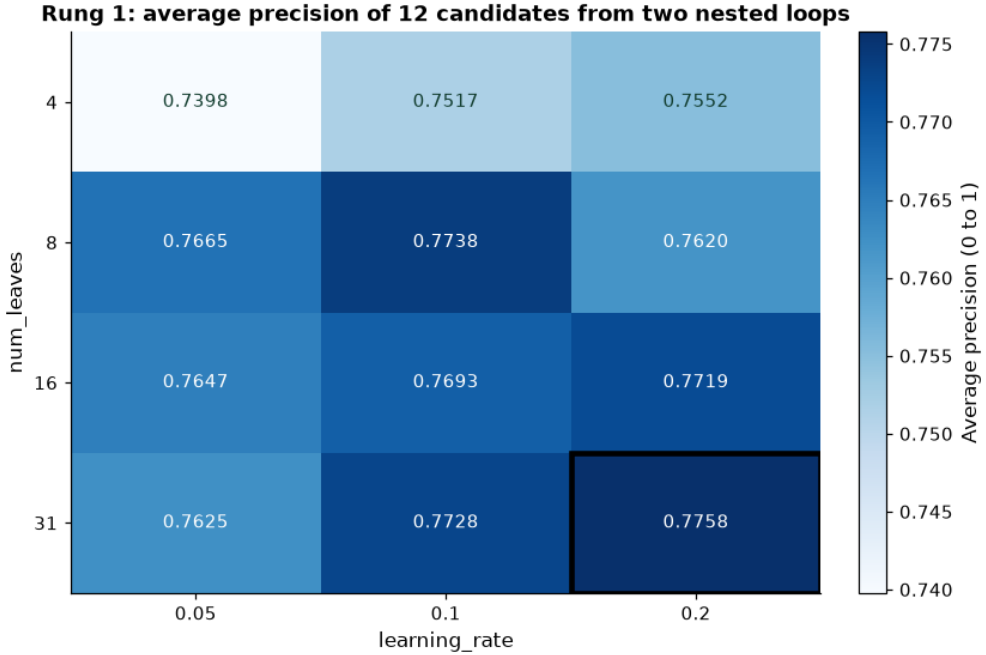
Five folds of the tuning rows score every rung. Each rung's winner is scored once on the check rows.

Setting	Values searched
`num_leaves`	4, 8, 16, 31
`learning_rate`	0.05, 0.1, 0.2
`min_child_samples`	default 20

Built from day 9's table of settings: the two that size the trees and set the step. The other tune-first groups stay at their defaults.

Two loops, twelve candidates





12 candidates and 60 fits, against 20 for one setting. The 4-leaf row is lowest in every column.

How GridSearchCV works

1. Lists

one list per setting

2. Every combination

the ParameterGrid of 11.1

3. Fit and score

each by the folds you give it

4. Refit the best

on all the rows it was given

You control

The lists, cv, scoring, and n_jobs (1 in the course).

What comes out

best_params_, best_score_, best_estimator_, and cv_results_.

How a randomized search works

1. Distributions

randint, loguniform

2. Draw n_iter

a fresh value of every setting

3. Fit and score

as the grid does

4. Keep the best

same outputs as the grid

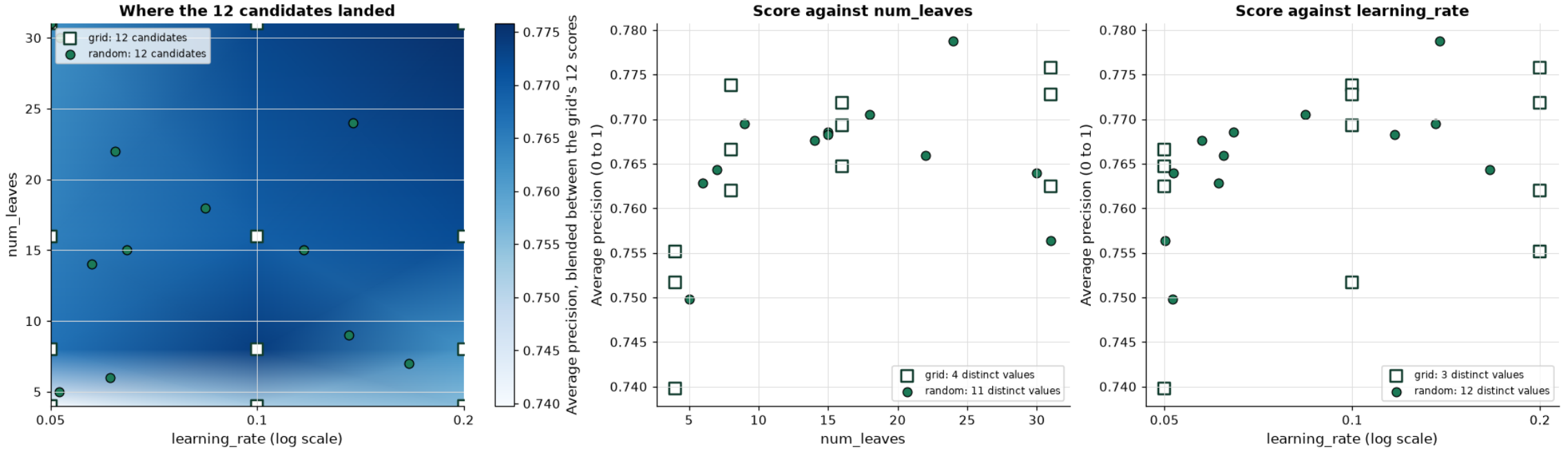
You control

The distributions, n_iter, the folds, and the seed.

What comes out

best_params_, best_score_, best_estimator_, and cv_results_.

Grid against random, on the same axes



Distinct values tried: 11 leaf counts and 12 rates for the random search, 4 leaf counts and 3 rates for the grid. Best 0.7788 against 0.7758: coverage is the claim.

How Optuna works

1. An objective

suggest_int, suggest_float(log=True)

2. A study calls it

again and again

3. TPE

random start, then draws where the better trials were dense

4. Pruning

the next step, not used here

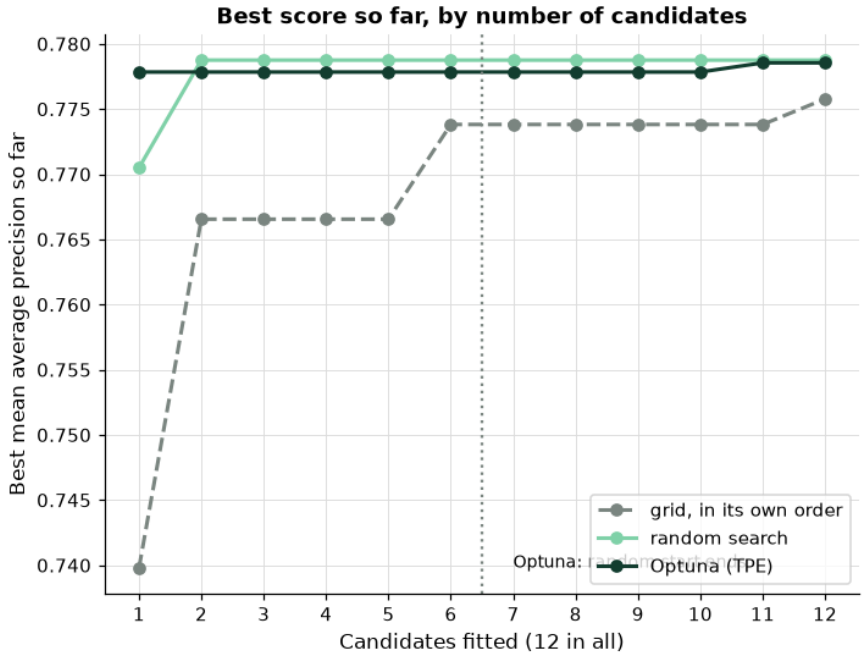
You control

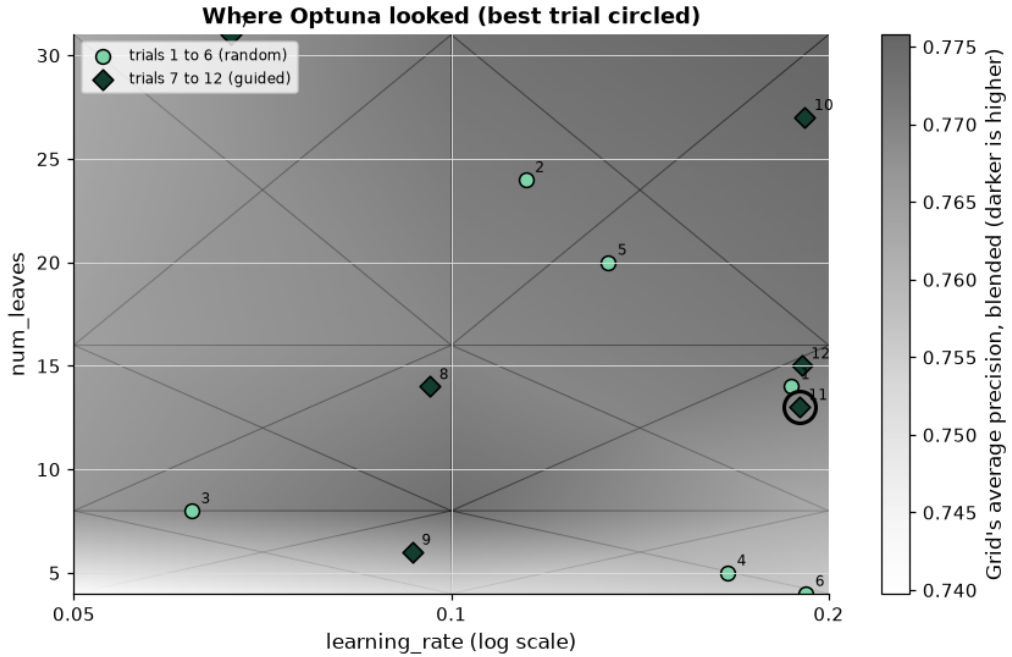
The ranges, n_trials, n_startup_trials, and the seed.

What comes out

best_params, best_value, and trials_dataframe().

Optuna against random and the grid

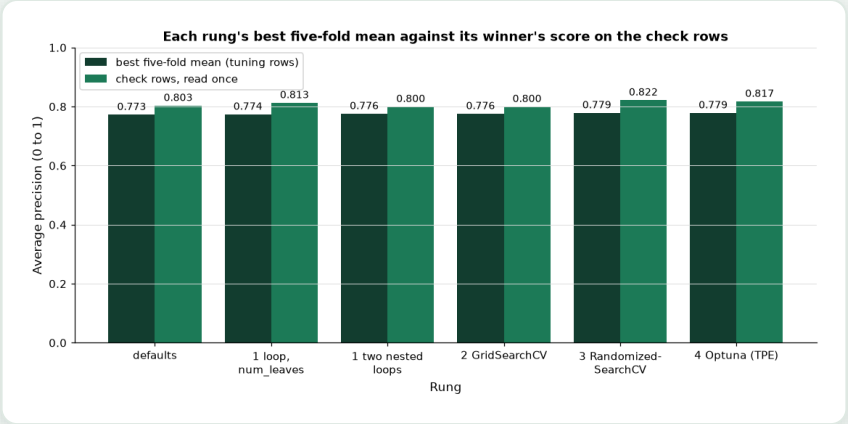




Optuna 0.7785 (best at trial 11), random 0.7788, grid 0.7758: inside one fold sd. Optuna 5.0.0, pinned.

The four rungs

Rung	Candidates	Fits	Best mean	Check rows
no search: the defaults	1	5	0.7728	0.8034
1 loop over num_leaves	4	20	0.7738	0.8130
1 two nested loops	12	60	0.7758	0.7998
2 GridSearchCV	12	60	0.7758	0.7998
3 RandomizedSearchCV	12	60	0.7788	0.8221
4 Optuna (TPE)	12	60	0.7785	0.8174



Cost counted in fits, not seconds. Check scores span 0.0222 with 65 bankrupt companies behind each: not a ranking.

The randomized search at full size

```
space = {"num_leaves": randint(8, 64),
         "learning_rate": loguniform(0.02, 0.3),
         "min_child_samples": randint(5, 60)}
```

n_iter = 6, under the grid's 8

6 candidates: up to 6 values of every setting, where the grid tries 2. random_state=SEED fixes the draws.

The search's best

num_leaves 46, learning_rate 0.1452, min_child_samples 25: best_score_ 0.7760

loguniform: 0.02 to 0.03 is as likely as 0.2 to 0.3.

Reading the search: means inside each other's noise

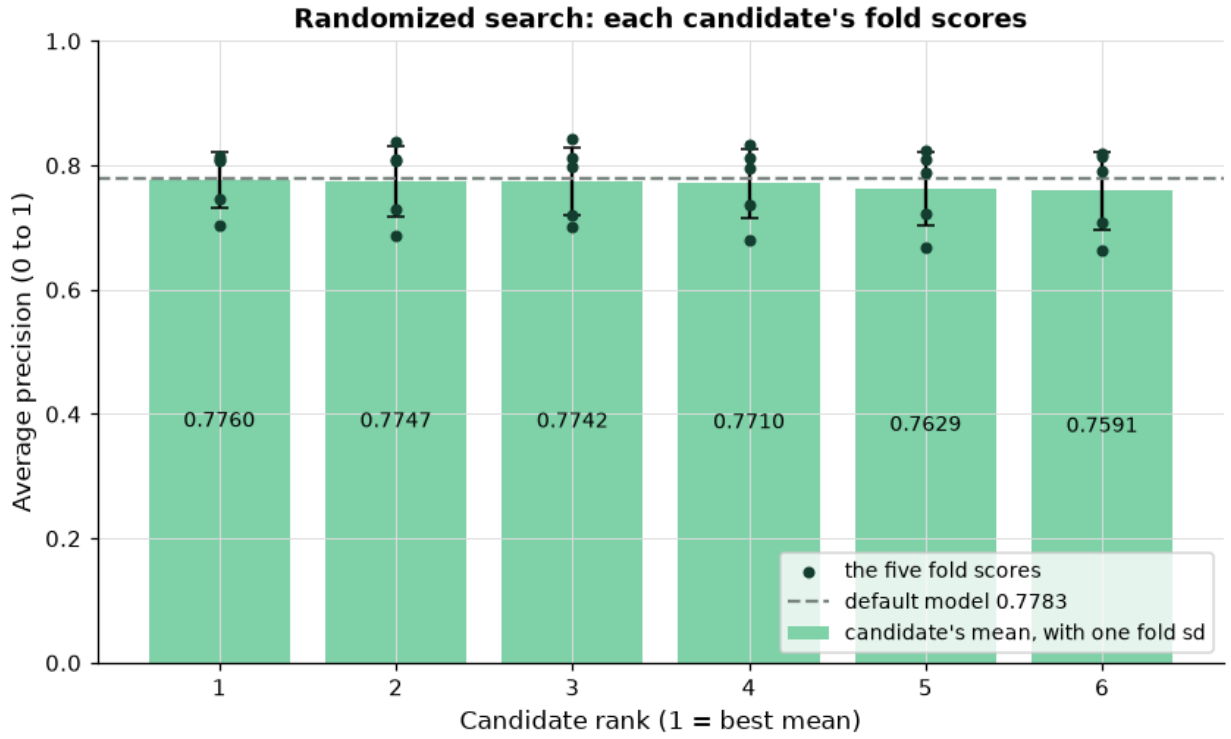
Rank	Mean	SD	Leaves	Rate	Rows/leaf
1	0.7760	0.0445	46	0.1452	25
2	0.7747	0.0564	22	0.0551	33
3	0.7742	0.0550	43	0.0694	57
4	0.7710	0.0561	18	0.0669	27
5	0.7629	0.0588	29	0.0294	7
6	0.7591	0.0622	51	0.0233	28

search_table

Every candidate's fold standard deviation is larger than the gap between any two neighbouring means.

42 tests pass after today's three functions.

Each candidate's fold scores



Folds against settings

0.0169

means apart

0.1130

least fold span

Fold 1 is the lowest dot of every candidate, and of the default model.

The folds make most of the spread; the settings a small part of it.

Tuning's gain against the folds' own noise

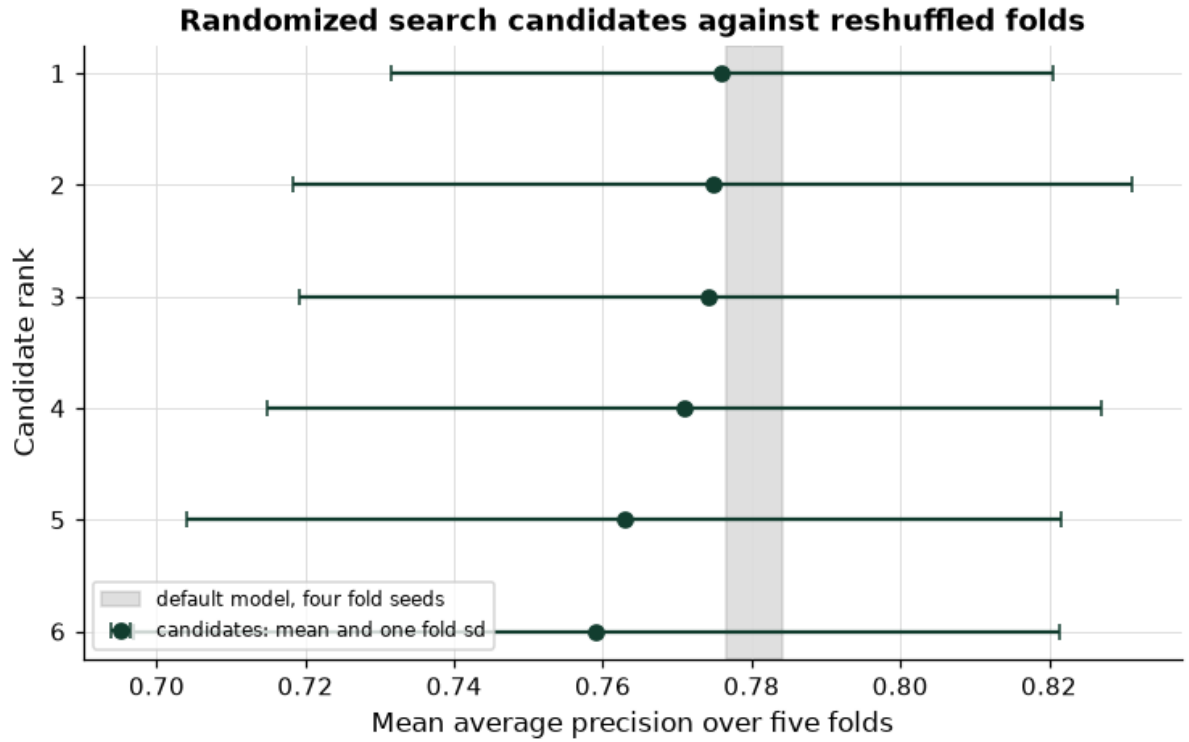
-0.0024

gain from tuning

0.0075

range, default on 4 fold seeds

Fold seed	Default mean
42	0.7783
43	0.7766
44	0.7841
45	0.7838



Tuning moves the mean less than reshuffling the folds does.

How the fold-seed check works

1. Fix model, rows

nothing about the model changes

2. Change the seed

a different cut into five folds

3. Score again

the same model on each cut

4. Take the range

the movement that comes from the cut alone

You control

The seeds: SEED and three after it.

What comes out

One mean per cut, and a range to hold a gain against.

How nested cross-validation works

1. Hold out

one outer fold

2. Search the rest

its own inner folds choose the winner

3. Refit and score

the winner on the held-out fold

4. Repeat

for each outer fold; average

You control

The outer folds, the inner folds, and the search.

What comes out

Five honest scores, whose mean is the score of the procedure.

Nested cross-validation: an honest score

1. Hold out

one outer fold

2. Search

on the other four, inner folds

3. Refit, score

on the held-out fold

4. Repeat

for all five

0.7697

nested mean

0.7760

best_score_

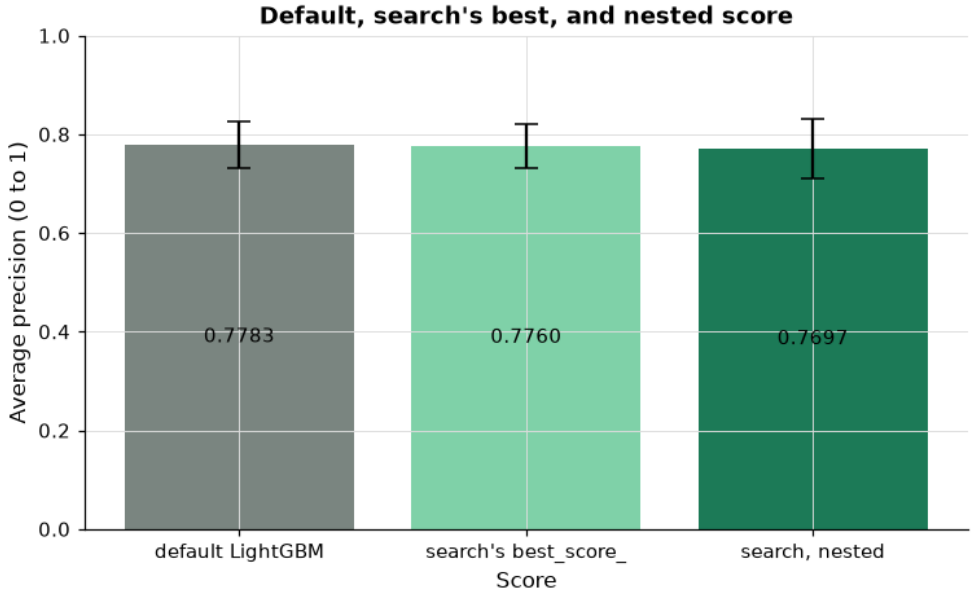
0.7783

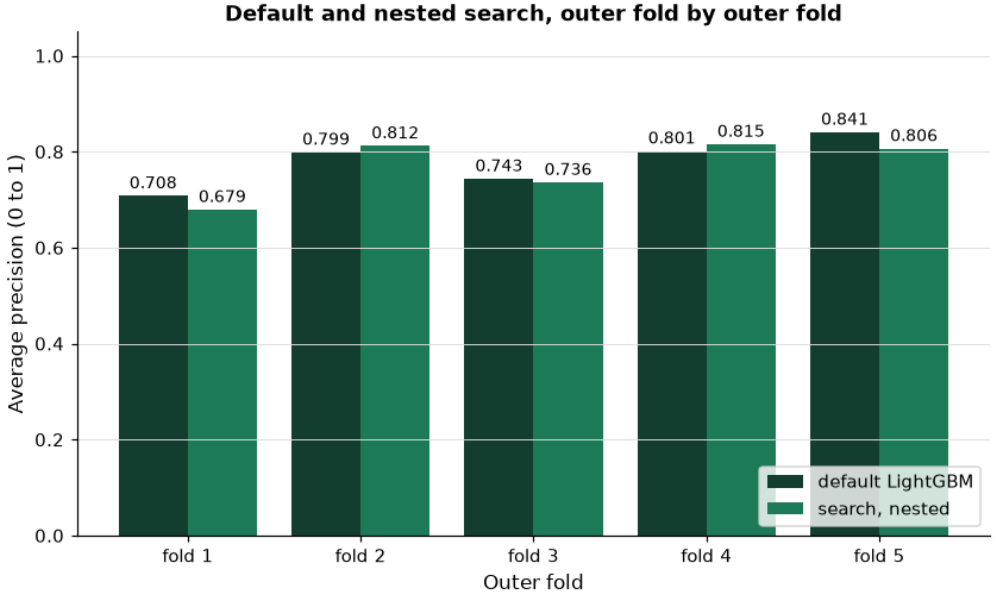
default model

Outer folds: 0.6792, 0.8120, 0.7358, 0.8153, 0.8062; sd 0.0603

Scored honestly, the tuned model does no better than the defaults on these rows.

Three scores, and the folds behind them





Score	Mean	Fold sd	Rows that chose the settings
default LightGBM	0.7783	0.0469	none chosen
search's best_score_	0.7760	0.0445	the same five folds
search, nested	0.7697	0.0603	inner folds only

Fold 1 hardest for both; nested higher on 2 of 5.

What to tune first, and when to stop

Defaults first	Score them: the baseline every search must beat
Tree size next	num_leaves or max_depth, rows per leaf
Then rate and trees	Together; early stopping picks the count
Fix the budget	n_iter and folds written before you look (n_iter=6)
Stop inside the noise	Gain below a fold sd or the reshuffle range

Practice that has served well, not a rule. Log every search with its nested score.

How the threshold is chosen

1. Out-of-fold

each from a model that did not see its row

2. The rule

the highest threshold with recall at least 0.70

3. One per model

outputs change scale with the settings

4. Flags

recall and precision at that threshold

You control

The recall target and the folds.

What comes out

A threshold per model, and its flags, recall, and precision.

The threshold is a choice too

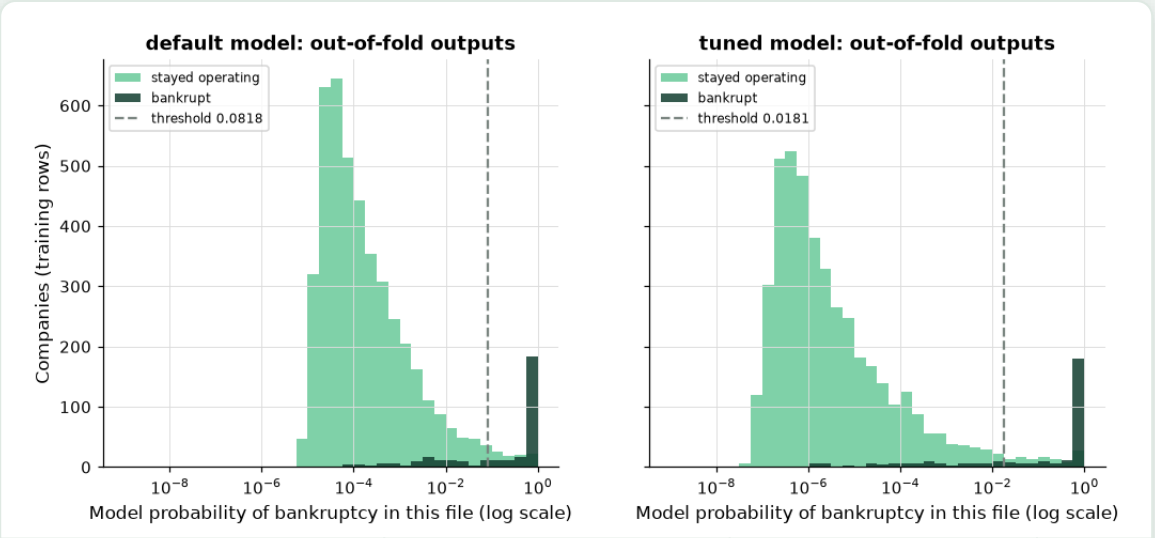
Model	Threshold	Flagged	Precision
Default	0.0818	327	0.7003
Tuned	0.0181	336	0.6815

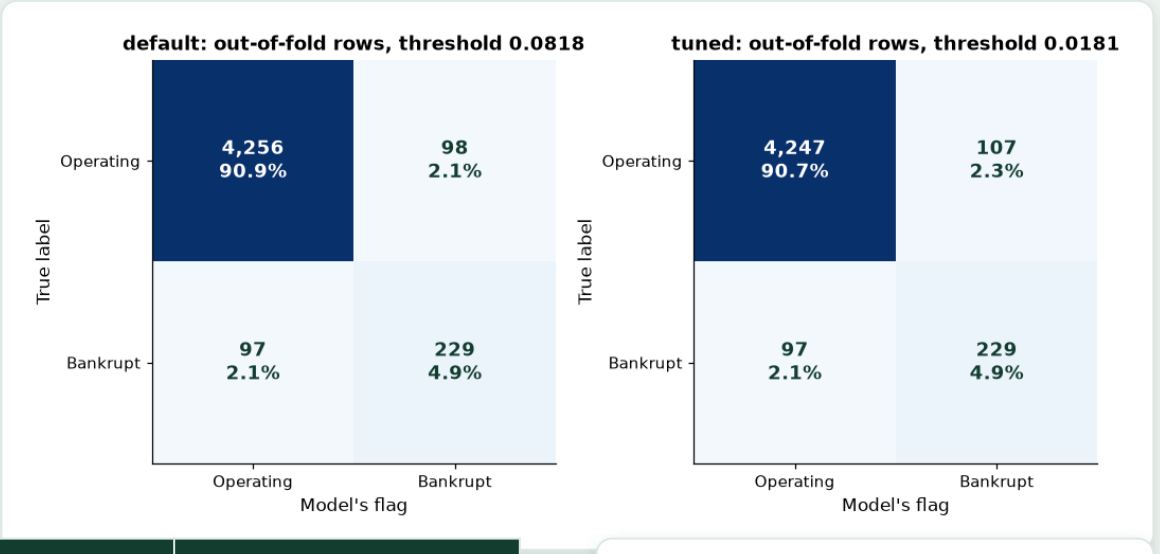
Recall at least 0.70 from out-of-fold probabilities, by Course 4's rule.

New settings change the scale of the outputs, so the threshold is chosen again after every search. The precision beside it is in-sample for the threshold.

Each output: the model probability of bankruptcy in this file. A flag is not a credit view.

Where the outputs fall, and the flags





Model	Threshold	Flagged	Recall	Precision
default	0.0818	327	0.7025	0.7003
tuned	0.0181	336	0.7025	0.6815

Median operating output: 1.03e-04 default, 1.39e-06 tuned

Both catch 229; false flags 98 and 107. Each model needs its own threshold.

Excel side by side

Excel side by side (tested in Excel)

`=AVERAGE(B2:F2)`	a candidate's mean, filled down
`=STDEV.P(B2:F2)`	std_test_score: 0.0445 for the best
`=STDEV.S(B2:F2)`	larger: 0.0497 (divides by 4)
`=MAX(G2:G7)`	best_score_ 0.7760
`=AVERAGE(A20:E20)`	nested mean 0.7697
`=MAX(A24:A27)-MIN(A24:A27)`	fold-seed range 0.0075

No randomized search and no cross-validation in Excel. It reads a search pasted onto a sheet.

Save the day: reading a commit

```
git show --stat HEAD
```

The last commit's message, then one line per file: its name, lines changed, and a bar of + for lines added.

```
model,cv_mean,cv_sd,nested_mean
lightgbm_default,0.7783,0.0469,0.7783
lightgbm_random_search,0.7760,0.0445,0.7697
```

A tuning day's log: best_score_ and the nested mean, side by side.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: best_score_ logged as the tuned model's honest score; the test asks for the nested mean.

Code that runs is not the same as code that is right.

What you can do now

- Rebuild Course 4's Polish rows; keep its test rows closed
- Loop, GridSearchCV, random search, Optuna: cost is candidates times folds
- A randomized search covers more values for the same budget
- Measure a gain against reshuffled folds
- Report the nested score beside best_score_

Week 3, Day 2 Feature engineering on financial ratios